Issue Tracker, end to end

A three-tier application carried from a laptop to a monitored Kubernetes deployment, and the delivery system built around it.

5	8	0	2
CI JOBS GREEN	TESTS PASSING	FIXABLE CVES	IMAGES SHIPPED

Rajveer Bishnoi · 24BCS10404 · Roll 10404

What it is

The application is deliberately small so the system around it can be the subject.

An issue has a title, a priority and an assignee, and moves through open, in progress and done. That is the whole feature set.

Everything the capstone is about — containers, CI, scanning, Helm, autoscaling, metrics, GitOps — wraps around those four verbs.

BOARD CENSUS, LIVE

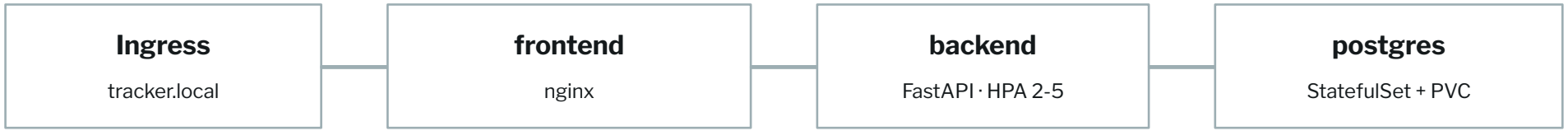
Census	5
Open	3
In progress	1
Done	1
Open · acuity 1	**2**

Architecture

DELIVERY PATH



RUNTIME, INSIDE THE CLUSTER



Prometheus discovers backend Pods by annotation, so a replica the autoscaler adds is scraped within one interval.

The Ingress is the only way in. postgres has no Service outside the namespace and no NodePort.

Stack

LAYER	PRINCIPAL	SUPPORTING
Application	React 18 · Vite	FastAPI · SQLAlchemy 2 · Alembic · PostgreSQL 16
Containers	Multi-stage	non-root UIDs · nginx-unprivileged · Docker Compose
Pipeline	GitHub Actions	GHCR · Trivy · Helm · kind inside the runner
Cluster	Minikube	ingress-nginx · metrics-server · one chart, three value files
Operations	Prometheus	Grafana · alert rules · Argo CD · Terraform for the network

Nothing exotic. The claim is that these fit together and were run, not that any one of them is clever.

The API

GET	/health	liveness · no dependencies
GET	/ready	readiness · checks the database
GET	/metrics	Prometheus exposition
GET	/api/issues	list, filterable
GET	/api/issues/stats	counts by status and priority
POST	/api/issues	create · validates priority
PUT	/api/issues/{id}	update
DELETE	/api/issues/{id}	delete

A liveness probe that queried the database would restart a healthy Pod every time the database blinked.

So /health answers from the process alone and /ready is the one that touches the database. The Pod leaves the Service instead of being killed.

That is what turned a bad config change into a stuck rollout rather than an outage.

Containers

```
# frontend: the toolchain never reaches the runtime image
FROM node:20-alpine AS build
FROM nginxinc/nginx-unprivileged:1.29-alpine
RUN apk upgrade --no-cache

# backend
FROM python:3.12-slim AS deps
FROM python:3.12-slim
RUN apt-get update && apt-get upgrade -y
```

```
$ docker compose up -d --build
  tracker-db        healthy
  tracker-backend   started   alembic upgrade head
  tracker-frontend  started   nginx proxies /api
```

Multi-stage

Node builds the bundle; nginx ships it. The compiler is not in the image that runs.

Non-root

A fixed non-root UID in both images, capabilities dropped in the manifests.

Upgraded bases

apk and apt upgrade at build time, so base-image CVEs do not ride along.

The pipeline

01

backend tests

pytest · JUnit artifact

01

frontend build

Vite · dist artifact

02

build, scan, push

buildx · Trivy · GHCR

03

deploy to kind

helm upgrade · smoke test

What the smoke test proves

It does not check that the release installed. It creates an issue through nginx, reads the stats back, and asserts the metrics counter moved — nginx, backend and postgres exercised in one pass.

```
NAME: tracker   STATUS: deployed   REVISION: 1
{"status":"ready"}
{"title":"created by CI","status":"open","id":1}
{"total":1,"by_status":{"open":1,"done":0}}
tracker_issues_created_total 0.0
```

Run 38892471244 on commit 17ee1b0 — five jobs green.

The gate

A scan that fails the build is only worth having if the fix is cheap.

Block on a vulnerability that clears with a version bump, because that fix costs one line. Report, but do not block, on one with no patch anywhere — a gate that stops work nobody can unblock is a gate people switch off.

```
severity:       HIGH,CRITICAL
ignore-unfixed: true
exit-code:      1
```

RESULT ON BOTH IMAGES

`issue-tracker-backend`	debian 13.7	**0**
`issue-tracker-frontend`	alpine 3.23.4	**0**

It failed twice before it passed: the frontend's Alpine base was behind, then Starlette under a pinned FastAPI. Both fixes were one-line version changes, and both are in the committed definitions.

Kubernetes and Helm

postgres	StatefulSet · volumeClaimTemplate · headless Service · pg_isready
backend	2 replicas · startup, readiness, liveness · runAsNonRoot · caps dropped
frontend	nginx serving the bundle · config from a ConfigMap proxying /api
Ingress	tracker.local · / to the frontend, /api to the backend
HPA	backend held between 2 and 5 replicas at 60% CPU

Persistence verified by deleting the postgres Pod: the StatefulSet rebound the same PVC and the issues survived.

Change the config and the Pods roll.

The backend Deployment carries a checksum of the ConfigMap and Secret as an annotation. Without it, a configuration change updates the objects and nothing restarts to pick it up.

```
annotations:
  checksum/config: {{ … }}
```

Observability

The autoscaler and the monitoring are designed together: neither has to know how many replicas there will be.

Chart annotates

prometheus.io/scrape on the backend

HPA adds a Pod

it carries the annotation already

Prometheus keeps it

discovered within one scrape interval

Grafana reads

rps by route, p95, issues created, error rate

ALERTS, BECAUSE A DASHBOARD NOBODY WATCHES CATCHES NOTHING

`BackendHighErrorRate`	5xx above five percent for two minutes
`BackendDown`	the target disappears

The app also counts issues created — a business signal, not an infrastructure one. It is what the CI smoke test asserts against.

GitOps

An Argo CD Application tracks the chart on main with automated sync, prune and self-heal. Change a value, commit, push; nobody runs helm.

GitOps and an autoscaler disagree about who owns the replica count.

The first version applied ignoreDifferences to every Deployment, so Argo CD also ignored a deliberate change to the frontend's replica count — the change was pushed, and nothing moved.

The conflict has to be declared, and declared for the right object.

```
ignoreDifferences:
  - group: apps
    kind: Deployment
    name: tracker-backend   # scoped
    jsonPointers:
      - /spec/replicas
```

Naming the HPA-managed Deployment is the whole fix.

Troubleshooting drill

Two faults introduced into the running release, each diagnosed from the cluster before any fix was applied.

ACUITY	FAULT	SYMPTOM	FOUND WITH	FIX
1 HIGH	**Wrong database host**	Rollout stuck, new Pod 0/1, old Pods still serving	kubectl logs, then describe probe events	**helm rollback**
2 MED	**Ingress to a Service that does not exist**	UI loads, every API call returns 503	describe ingress named the missing Service	**re-render the Ingress**

The first fault is why the probes are shaped the way they are: readiness checks the database, so the new Pod never became Ready and the rolling update refused to proceed. The old Pods kept serving.

What it taught

01 A readiness probe that checks the database turns a bad config change into a stuck rollout instead of an outage.

02 A scan that fails the build is useful only where the fix is cheap — so block on fixable findings and report the rest.

03 GitOps and autoscaling both claim the replica count, and the conflict has to be declared narrowly.

04 Running the whole stack locally first made the cloud step a change of target, not a different project.